Homework 5

Comments are required for all parts of this homework

Problem 1: Faceted Histogram

Run the following code block to define a function which generates two 1-dimensional `numpy` arrays. The first array, called `groups`, consists of integers between `0` and `n_groups - 1`, inclusive. The second array, called `data`, consists of real numbers.

In [42]:
```
import numpy as np
from matplotlib import pyplot as plt

def create_data(n, n_groups):
    """
    generate a set of fake data with group labels.
    n data points and group labels are generated.
    n_groups controls the number of distinct groups.
    Returns an np.array() of integer group labels and an
    np.array() of float data.
    """

    # random group assignments as integers between 0 and n_groups-1, inclusive
    groups = np.random.randint(0, n_groups, n)

    # function of the groups plus gaussian noise (bell curve)
    data   = np.sin(groups) + np.random.randn(n)

    return(groups, data)
```

Part A

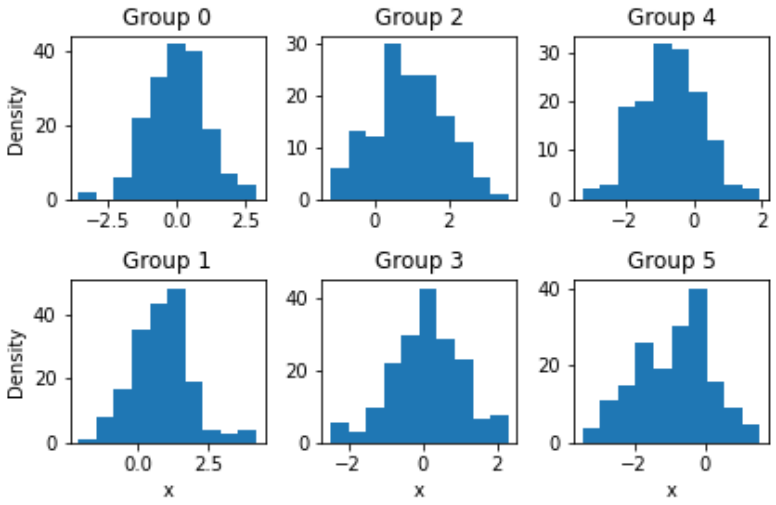
Write a function called `facet_hist()`. This function should accept five arguments:

1. `groups`, the `np.array` of group labels as output by `create_data()`.
2. `data`, the `np.array` of data as output by `create_data()`.
3. `m_rows`, the number of desired rows in your faceted histogram (explanation coming).
4. `m_cols`, the number of desired columns in your faceted histogram (explanation coming).
5. `figsize`, the size of the figure.

Your function will create faceted histograms -- that is, a separate axis and histogram for each group. For example, if there are six groups in the data, then you should be able to use the code

```
groups, data = create_data(1000, 6)
facet_hist(groups, data, m_rows = 2, m_cols = 3, figsize = (6,4))
```

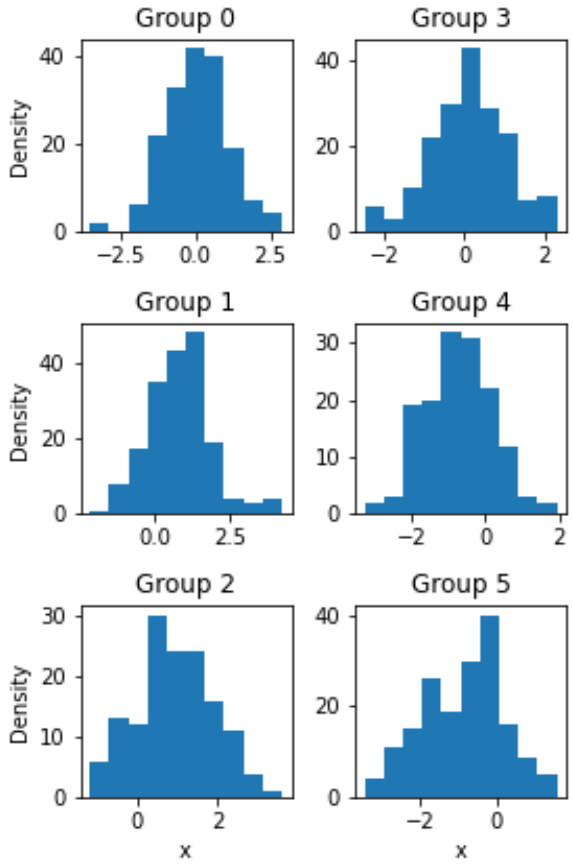
to create a plot like this:



It's fine if your group labels run left-to-right (so that the top row has labels 0, 1, and 2 rather than 0, 2, 4).

You should also be able to change the orientation by modifying `m_rows`, `m_cols`, and `figsize`.

```
facet_hist(groups, data, m_rows = 3, m_cols = 2, figsize = (4,6))
```



Requirements:

1. Your function should work **whenever `m_rows*m_cols` is equal to the total number of groups.** Your function should first check that this is the case, and raise an informative `ValueError` if not. You may assume that there is at least one data point for each group label in the data supplied.
2. For full credit, you should not loop over the individual entries of `groups` or `data`. It is acceptable to loop over the distinct values of `groups`. In general, aim to minimize `for`-loops and maximize use of `Numpy` indexing.
3. Use of `pandas` is acceptable but unnecessary, and is unlikely to make your solution significantly simpler.
4. You should include a horizontal axis label (of your choice) along **only the bottom row** of axes.
5. You should include a vertical axis label (e.g. "Frequency") along **only the leftmost column of axes.**
6. Each axis should have an axis title of the form "Group X", as shown above.
7. Comments and docstrings!

Hints

- If your plots look "squished," then `plt.tight_layout()` is sometimes helpful. Just call it after constructing your figure, with no arguments.
- Integer division `i // j` and remainders `i % j` are helpful here, although other solutions are also possible.

In [46]:
```
# your solution here
def facet_hist(groups,data,m_rows,m_cols,figsize,**kwargs):
    """
    Creates faceted histograms, where there is a separate axis and histogram for each group.
    Only works when the number of desired rows in histogram equals the number of desired columns.
    Otherwise, function raises an informative ValueError.

    parameter 1, groups: an np.array of group labels, output by function create_data()
    parameter 2, data: an np.array of data, output by function create_data()
    parameter 3, m_rows: number of desired rows in your faceted histogram
    parameter 4, m_cols: number of desired columns in your faceted histogram
    parameter 5, figsize: the size of the whole figure, tuple of width and height in inches

    Also accepts kwargs, any arbitrary number of keyword arguments inputted by the user.

    """
    n_groups = np.unique(groups) #counting number of distinct groups in groups so we can loop over them later

    fig,ax=plt.subplots(m_rows,m_cols,figsize = figsize) #creating a plot of subplots with number of rows,
                                                         #columns, and figsize as specified by user

    if m_rows*m_cols != len(n_groups): #if number of rows*cols does not equal the number of unique groups
        raise ValueError ("Number of rows must be equal to number of columns.") #raise informative ValueError
    else: #else when rows*cols equals the number of distinct groups
        for i in n_groups:              #looping over distinct groups
            graph_data=data[groups==i] #getting the corresponding data of the group and
                                       #assigning it to graph_data, to be plotted later

            #assigning the group a subplot
            row = i//m_cols #assigning the group to an appropriate subplot row in the plot
            col = i%m_cols  #assigning the group to the correct subplot column in the plot

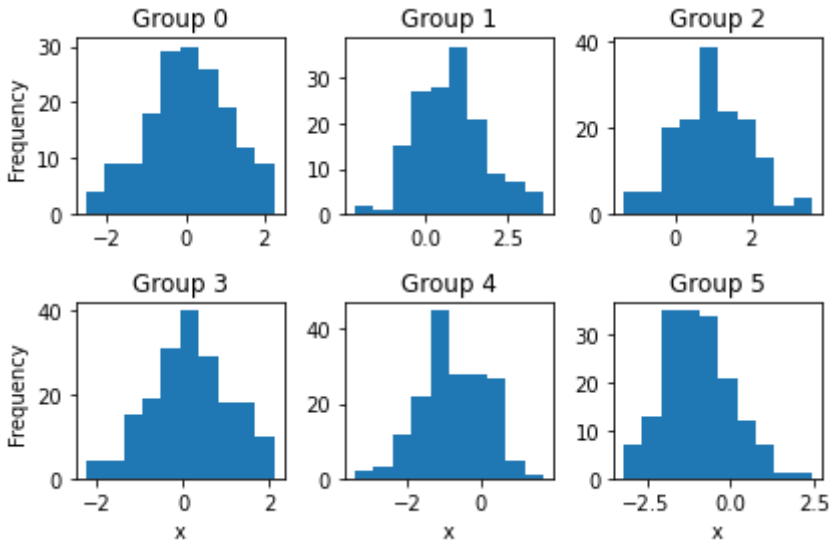
            ax[row,col].hist(graph_data,**kwargs) #graphing a histogram with the corresponding data
                                                  #in the correct row/col subplot and
                                                  #passing appropriate keyword arguments, if any
            ax[row,col].set(title="Group "+str(i)) #titling subplot as appropriate Group number
```

```
            if row==m_rows-1: #labeling the last row's x-axes 'x'
                ax[row,col].set(xlabel="x")

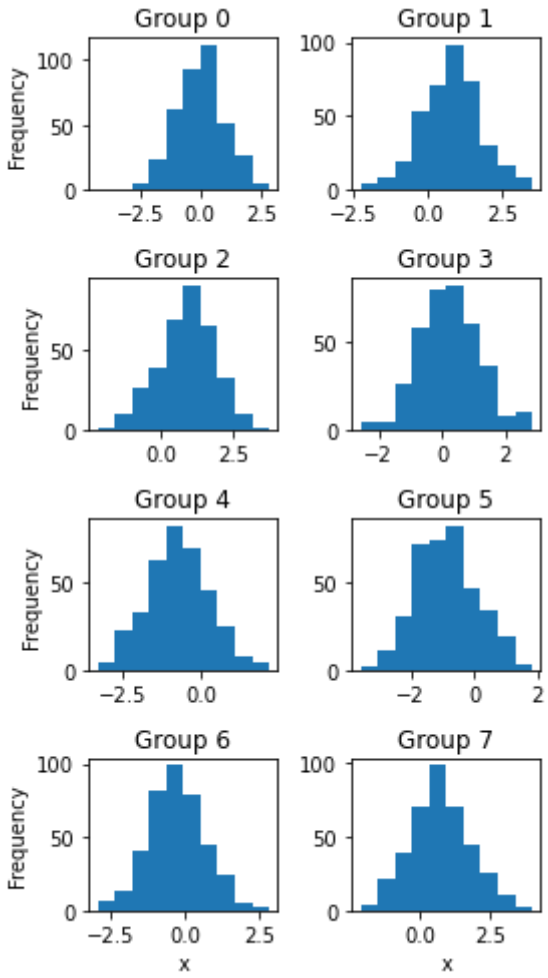
            if col==0: #labeling the leftmost column's y-axes 'Frequency'
                ax[row,col].set(ylabel="Frequency")

        plt.tight_layout() #making graph look nicer
```

In [47]:
```
# test code
groups, data = create_data(1000, 6)
facet_hist(groups, data, 2, 3, figsize = (6, 4))
```



In [48]:
```
# test code
groups, data = create_data(3000, 8)
facet_hist(groups, data, 4, 2, figsize = (4, 7))
```

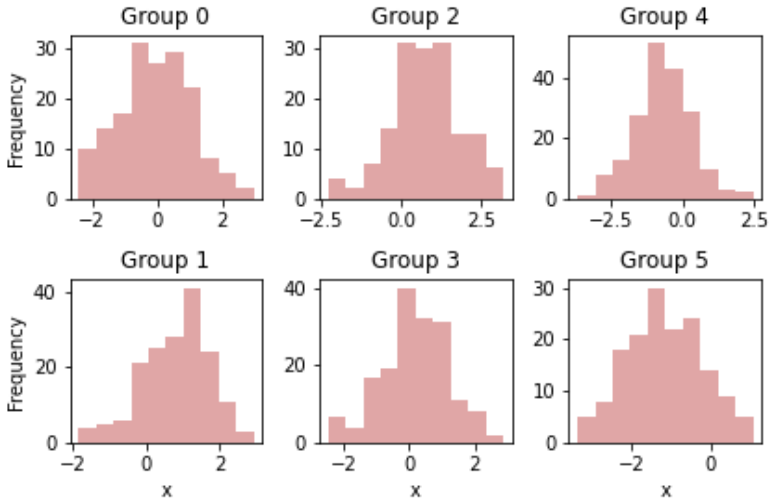


Part B

Modify your function (it's ok to modify it in place, no need for copy/paste) so that it accepts additional `**kwargs` passed to `ax.hist()`. For example,

```
facet_hist(groups, data, 2, 3, figsize = (6, 4), alpha = .4, color = "firebrick")
```

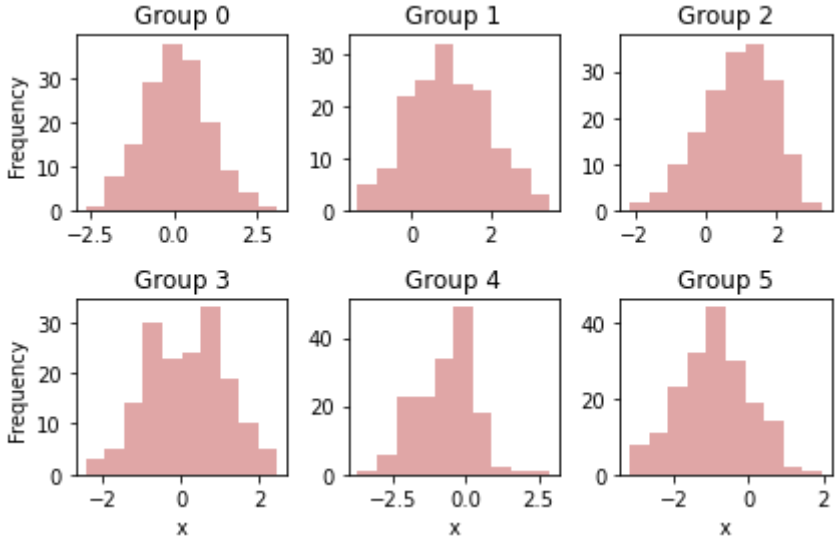
should produce



Example output.

You should be able to run this code **without defining parameters `alpha` and `color` for `facet_hist()`.**

In [49]:
```
# run this code to show that your modified function works
groups, data = create_data(1000, 6)
facet_hist(groups, data, 2, 3, figsize = (6, 4), alpha = .4, color = "firebrick")
```



Problem 2: Scatterplot Matrices

Run the following code to download, import, and display a data set from the 2019 World Happiness Report.

In [50]:
```
# if you experience ConnectionRefused errors, you may instead
# copy the url into your browser, save the file as data.csv
# in the same directory as the notebook, and then replace the
# third line with
# happiness = pd.read_csv("data.csv")

import pandas as pd
url = "https://philchodrow.github.io/PIC16A/datasets/world_happiness_report/2019.csv"
happiness = pd.read_csv(url)
happiness
```

Out[50]:

	Overall rank	Country or region	Score	GDP per capita	Social support	Healthy life expectancy	Freedom to make life choices	Generosity	Perceptions of corruption
0	1	Finland	7.769	1.340	1.587	0.986	0.596	0.153	0.393
1	2	Denmark	7.600	1.383	1.573	0.996	0.592	0.252	0.410
2	3	Norway	7.554	1.488	1.582	1.028	0.603	0.271	0.341
3	4	Iceland	7.494	1.380	1.624	1.026	0.591	0.354	0.118
4	5	Netherlands	7.488	1.396	1.522	0.999	0.557	0.322	0.298
...	...	...	...	...	...	...	...	...	...
151	152	Rwanda	3.334	0.359	0.711	0.614	0.555	0.217	0.411
152	153	Tanzania	3.231	0.476	0.885	0.499	0.417	0.276	0.147
153	154	Afghanistan	3.203	0.350	0.517	0.361	0.000	0.158	0.025
154	155	Central African Republic	3.083	0.026	0.000	0.105	0.225	0.235	0.035
155	156	South Sudan	2.853	0.306	0.575	0.295	0.010	0.202	0.091

156 rows × 9 columns

This is a `pandas` data frame. Observe the following:

1. Each row corresponds to a country or region.

2. The `Score` column is the overall happiness score of the country, evaluated via surveys.
3. The other columns give indicators of different features of life in the country, including GDP, level of social support, life expectancy, freedom, generosity of compatriots, and perceptions of corruption in governmental institutions.

   You can extract each of these columns using dictionary-like syntax:

```
happiness["Score"]
```

```
0      7.769
1      7.600
2      7.554
3      7.494
4      7.488
     ...
151    3.334
152    3.231
153    3.203
154    3.083
155    2.853
Name: Score, Length: 156, dtype: float64
```

Technically, this output is a `pandas` `Series`; however, in this context (and most others) it's fine to simply think of it as a 1-dimensional `np.array()`.

Part A

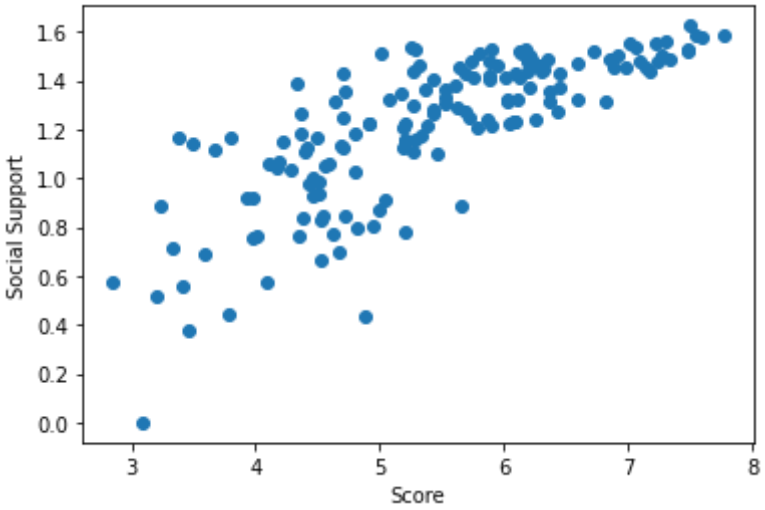
As a warmup, create a scatterplot of the overall `Score` column against a numerical column of your choice. Give the horizontal and vertical axes appropriate labels. Discuss your result. Is there a correlation? Does that correlation make sense to you?

In [51]:
```
# plotting code here

fig,ax=plt.subplots(1) # creating the base of the plot
ax.scatter(happiness['Score'],happiness['Social support'])
# plotting happiness score against data from the social support column on scatterplot
# score on x-axis, social support on y-axis

ax.set(xlabel="Score",ylabel="Social Support")
# labeling the axes according to above
```

Out[51]:
```
[Text(0.5, 0, 'Score'), Text(0, 0.5, 'Social Support')]
```



Yes, there seems to be some level of correlation. Score and social support seem to have a positive correlation. In other words, as the amount of social support increases, so does the happiness score. This does make sense to me since more social support means people have less financial struggles (which is a significant source of stress for many), overall making them happier.

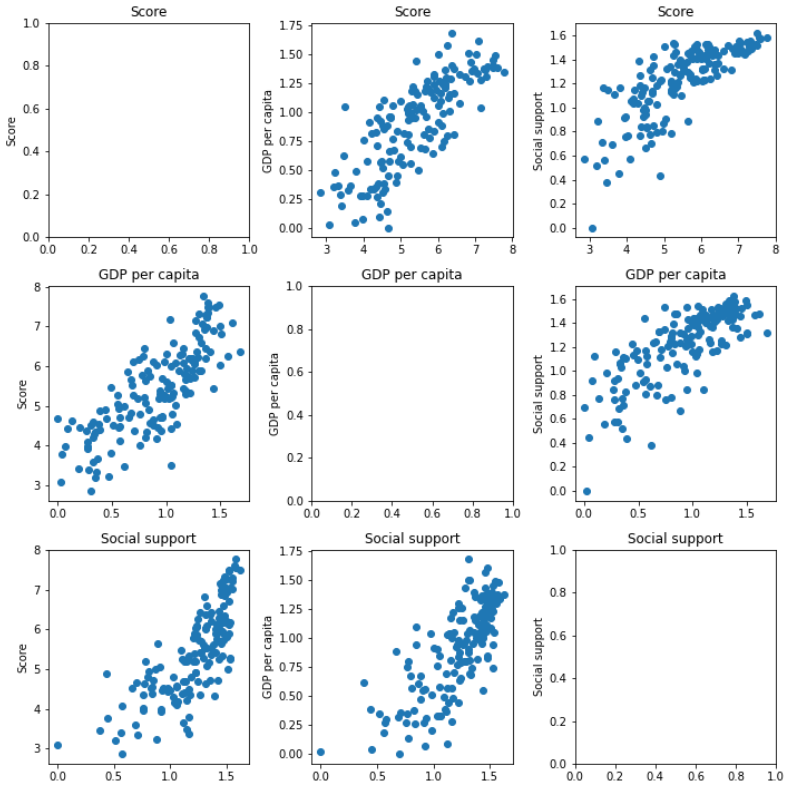
Part B

That plot you made may have helped you understand whether or not there's a relationship between the overall happiness score and the variable that you chose to plot. However, there are several variables in this data set, and we don't want to manually re-run the plot for each pair of variables. Let's see if we can get a more systematic view of the correlations in the data.

Write a function called `scatterplot_matrix()`, with arguments `cols` and `figsize`. The `cols` argument should be a list of strings, each of which are the name of one of the columns above, for example `cols = ["Score", "GDP per capita", "Social support"]`. Your function should create a *scatterplot matrix*, like this:

```
cols = ["Score",
        "GDP per capita",
        "Social support"]

scatterplot_matrix(cols,figsize = (7,7))
```



There is a separate scatterplot for each possible pair of variables. In fact, there are two: one where the first variable is on the horizontal axis, and one where it's on the vertical axis. Some analysts prefer to remove half the plots to avoid redundancy, but you don't have to bother with that. The diagonal is empty, since there's no point in investigating the relationship between a variable and itself.

Don't forget comments and docstrings!

In [65]:
```
# define your function
def scatterplot_matrix(cols, figsize):
    """
    Plots every possible variable pair combination from the happiness dataframe in a separate scatterplot.
    Makes two scatterplots for every pair, one with first variable on x-axis, one where it is on the y-axis.
    Keeps plots of same variable compared to itself blank.

    parameter 1, cols: list of strings of the different columns in the dataframe
        (or variables for the scatterplots)
    parameter 2, figsize: size of the scatterplot, tuple of width and height in inches
```

```
    """

    fig,ax=plt.subplots(len(cols),len(cols),figsize=figsize) # creating plot of subplots with as many rows
                                                             # and columns as there are variables in cols
                                                             # since each variable is on each axis once
                                                             # figure size is the figsize specified by user
    for i in range(len(cols)):     # looping over the variables in cols
        for k in range(len(cols)): # nested for loop looping over variables in cols again separately
                                   # so each variable/variable pair is considered
            if i!=k: # only graph plot if current first and second for loop's variables are not the same,
                     # since the two same variable graphs should be blank
                ax[i,k].scatter(happiness[cols[i]], # plotting the current variable from cols against
                                happiness[cols[k]]) # the other one from cols, on a scatterplot
                my_number = (np.corrcoef(       # finding the correlation coefficient of the current
                    happiness[cols[i]],         # variable/variable pair
                    happiness[cols[k]])[1][0])  # since this returns an array, we need the [1,0] index

                ax[i,k].set(xlabel = r"$\rho$ = " + str(  # adding corresponding correlation coefficient
                    np.round(my_number, 2)))              # rounded to 2 decimal places to the x-axis

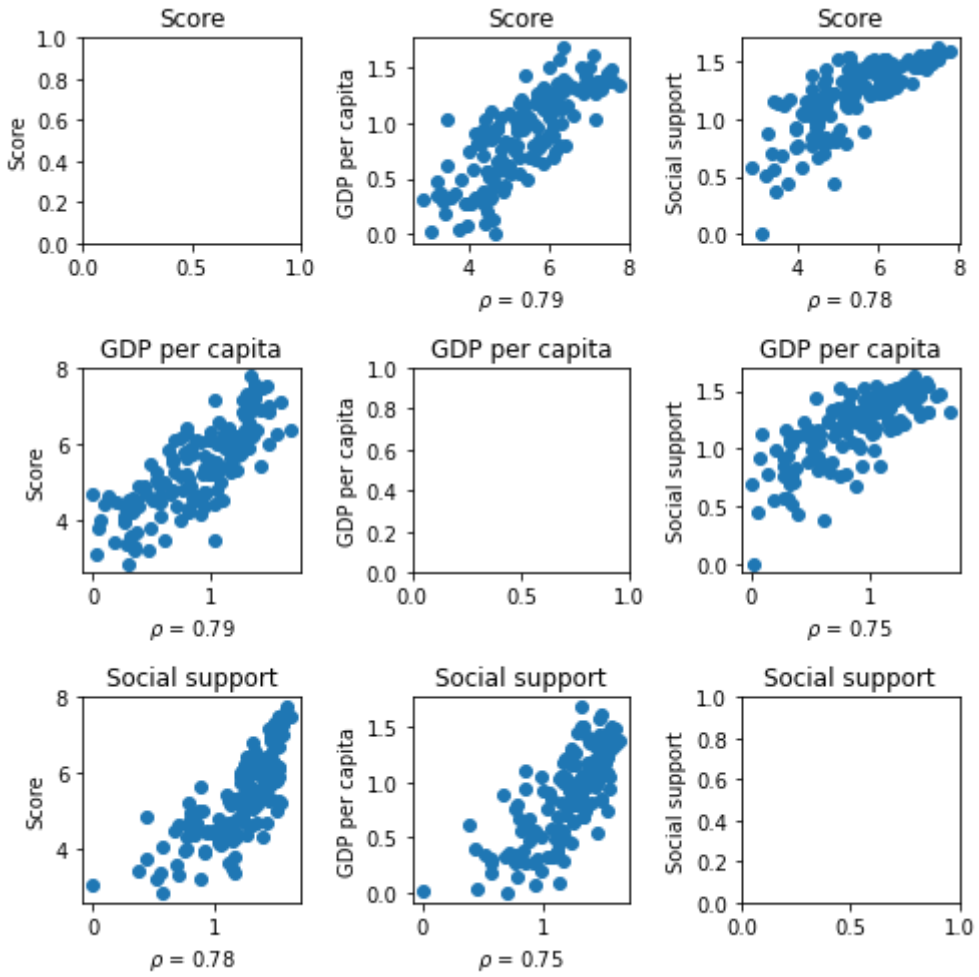
            ax[i,k].set(ylabel=cols[k], # label y-axis the name of the variable from first loop that it is
                    title=cols[i])      # label top x-axis name of the current variable from second loop

    plt.tight_layout()  # making the scatterplots look nicer, adjusting formatting
```

In [55]:
```
# test your code, several times if needed, and discuss the correlations you observe.
# Add code cells if needed to show multiple outputs.

cols = ["Score",
        "GDP per capita",
        "Social support"]

scatterplot_matrix(cols,figsize = (7,7))
```



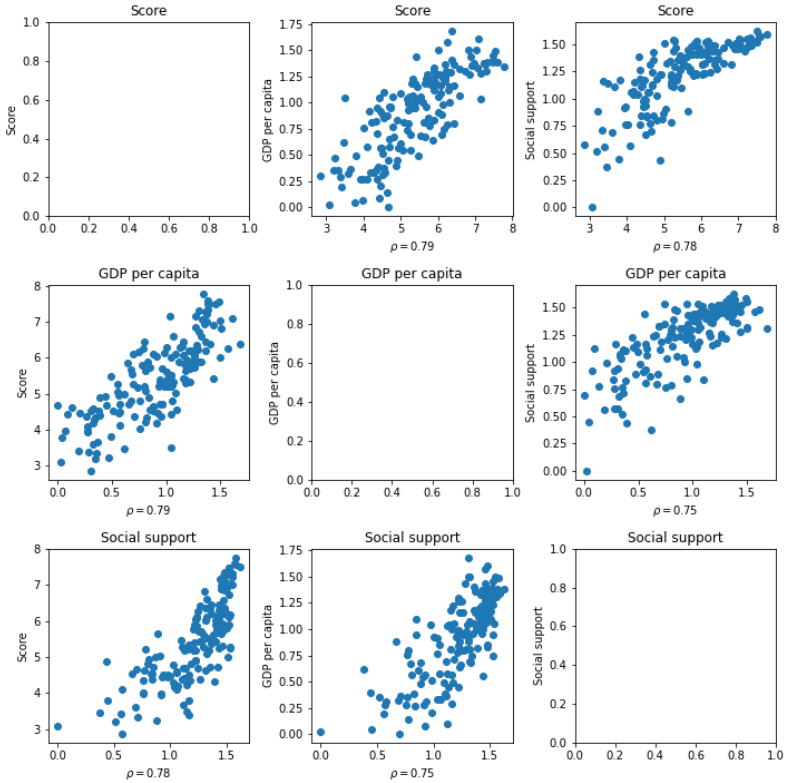
Part C

The *correlation coefficient* is a measure of linear correlation between two variables. The correlation coefficient between X and Y is high if X tends to be high when Y is, and vice versa. Correlation coefficients lie in the interval $[-1, 1]$.

`numpy` provides a function to conveniently compute the correlation coefficient between two or more variables. Find it, and then use it to add "captions" (as horizontal axis labels) to each panel of your plot giving the correlation coefficient between the plotted variables. For example,

```
cols = ["Score",
        "GDP per capita",
        "Social support"]

scatterplot_matrix(cols,figsize = (7,7))
```

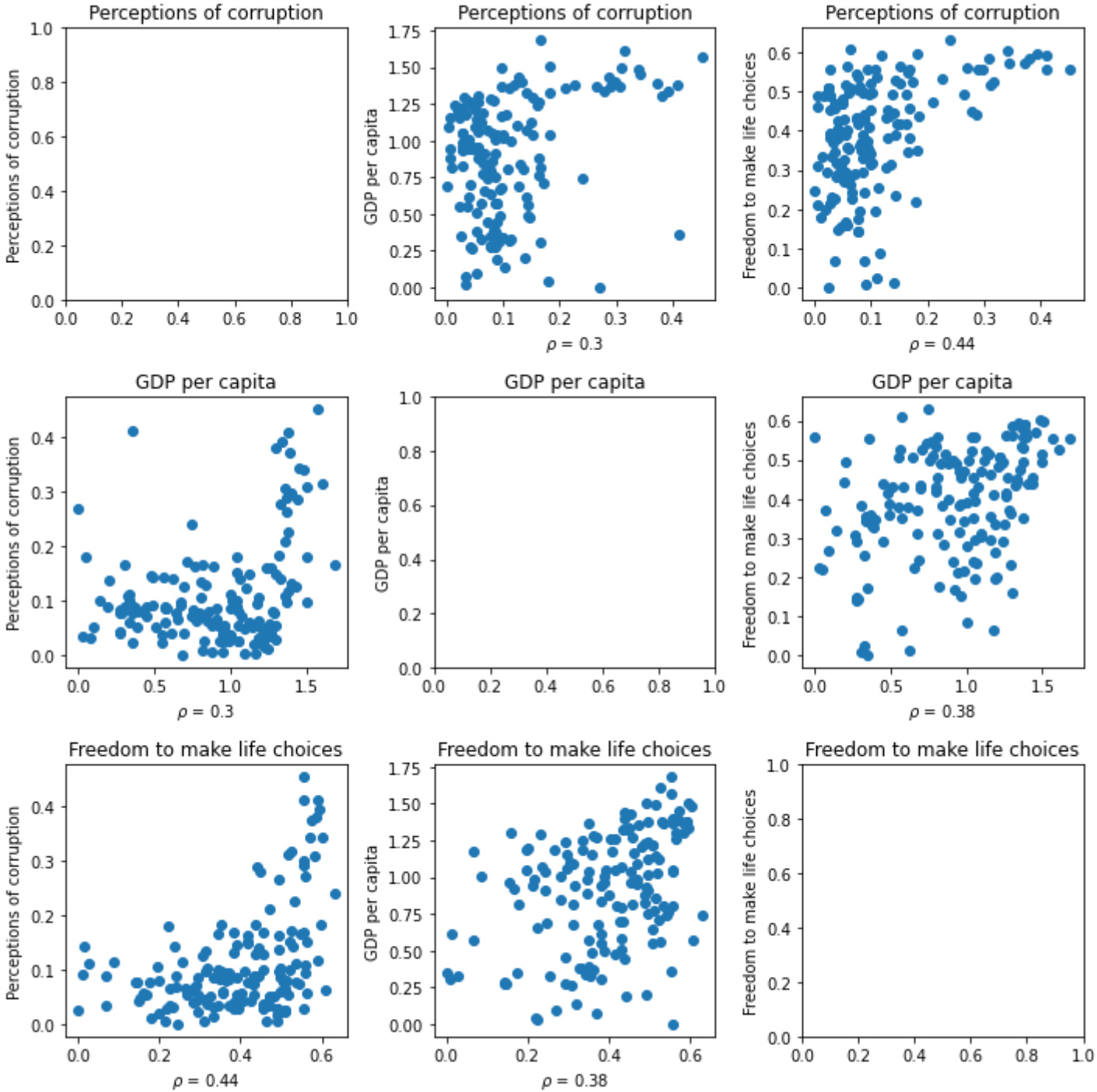


It's not required that you add the Greek letter ρ (the classical symbol for correlation coefficients), but if you do want to, here's how. You can also tweak the rounding as desired.

```
ax.set(xlabel = r"$\rho$ = " + str(np.round(my_number, 2)))
```

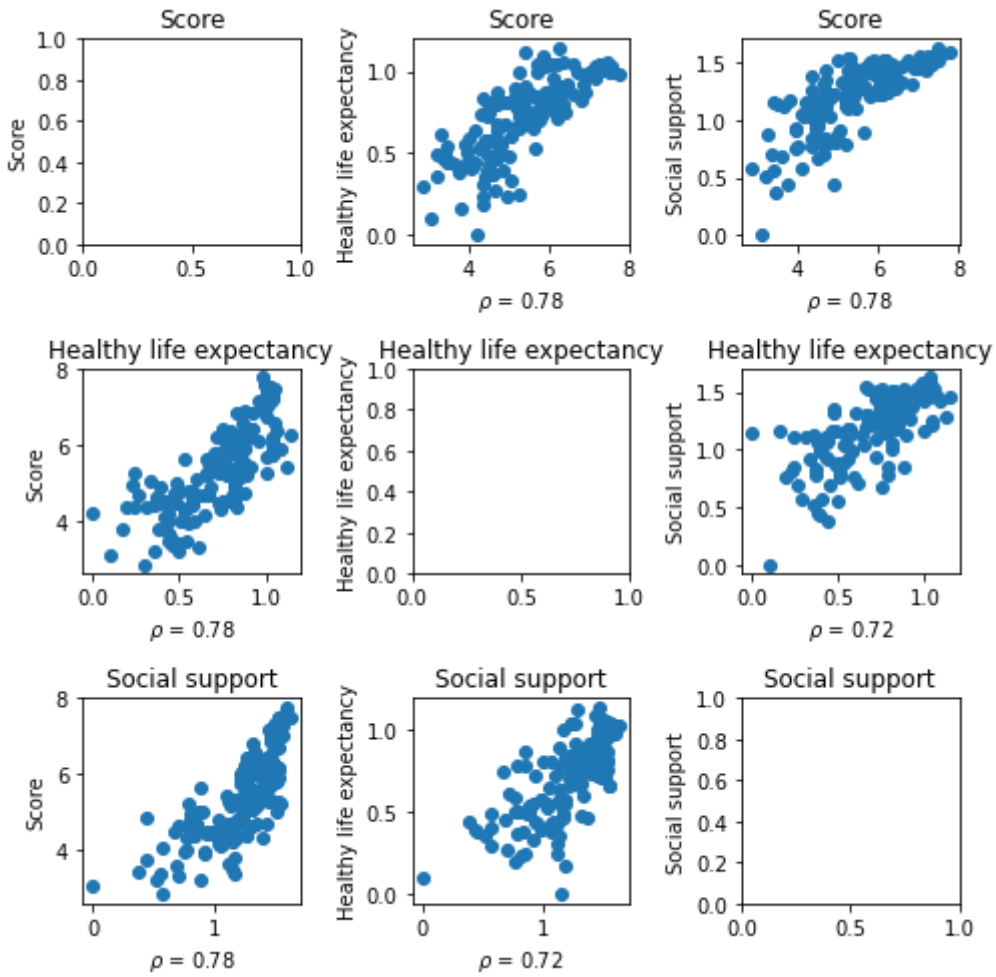
In [56]:
```
#testing function for corruption, GDP, and freedom columns
#increased fig size due to long column names
cols = ["Perceptions of corruption",
        "GDP per capita",
        "Freedom to make life choices"]

scatterplot_matrix(cols,figsize = (10,10))
```


Perceptions of corruption
Perceptions of corruption
GDP per capita
ρ = 0.3
Perceptions of corruption
Freedom to make life choices
ρ = 0.44
GDP per capita
Perceptions of corruption
ρ = 0.3
GDP per capita
GDP per capita
Freedom to make life choices
ρ = 0.38
Freedom to make life choices
Perceptions of corruption
ρ = 0.44
Freedom to make life choices
GDP per capita
ρ = 0.38
Freedom to make life choices
Freedom to make life choices

In [57]:
```
#testing function for score, life expectancy, and social support
cols = ["Score",
        "Healthy life expectancy",
        "Social support"]

scatterplot_matrix(cols,figsize = (7,7))
```



Run your code on several different subsets of the columns. It's ok to simply re-run your Part B results where they are and show the output including the correlation coefficient. Discuss your findings. What positive correlations do you observe? Do they make sense? Are there any negative correlations? Do the quantitative results match what you see "by eye"?

If you were going to create a model to attempt to predict overall happiness from other indicators, which columns would you use? Why?

In the first run of the function for columns score, GDP, and social support, I see various positive correlations. All variable pairs here are positively correlated, with the strongest positive correlations being happiness score and GDP and GDP and happiness score. This makes sense because more social support lessens burdens on people, making them happier. It also makes sense that higher GDP is correlated with a higher happiness score for a country, since people tend to be happier when the economy is doing better (more public goods and less unemployment usually). Higher GDP and higher social support also makes sense since a better off economy means the government has more resources to help people. These quantitative results do match what I see by eye, I can clearly see a stronger positive correlation than the other graphs.

In the second run of the function for columns perceptions of corruption, GDP, and freedom to make life choices, there are much weaker positive correlations. These make less sense. I would think that when there are higher perceptions of corruption, the GDP is lower and freedom is also lower, since leaders would do more for their benefit (not the economy) and corrupt leaders usually limit citizens' freedom to limit opposition. The GDP and freedom one makes sense, because I would think that when the economy is doing well people have more freedom to make choices especially because they probably have more money to make choices. These quantitative results do not fully match what I see by eye. Some of the graphs like gdp vs. freedom look like there is very little to no correlation, I thought the correlation coefficient would be even smaller. However, in terms of positive or negative correlation they do match what I see by eye.

In the last run with columns life expectancy, social support, and happiness score, there are strong positive correlations. This makes sense since more social support means people have access to things they need to extend their lives and maintain their wellbeing. Social support and happiness score being positively correlated also makes sense since people will have to deal with less hardships and stress. Health is also normally correlated with overall happier people due to science, so happiness score increasing with healthy life expectancy makes sense too. The quantitative results do match the correlations that I see by eye.

Overall, none of the scatterplots I made showed negative correlations.

To predict happiness, I would use columns such as GDP per capita, healthy life expectancy, and social support because they all have pretty strong correlations with happiness score in our graphs (around 0.78-0.79), showing that there is definitely some relationship there.

Problem 3: Plotting Time Series

Run the following code to download two time series data sets:

- Historical data on the Dow Jones Industrial Average (a composite performance measure of the US stock market), retrieved from Yahoo Finance.
- Cumulative COVID19 cases over time, from the New York Times.

In [62]:
```
# run this block
# if you experience ConnectionRefused errors, you may instead
# copy the urls into your browser, save the files as DJI.csv
# and COVID.csv respectively in the same directory as the notebook.
# Then, in the lines using the function pd.read_csv(), replace
# the url with "DJI.csv" and "COVID.csv"

import pandas as pd
import datetime

url = "https://query1.finance.yahoo.com/v7/finance/download/%5EDJI?period1=1580750232&period2=1712372632&inte
DJI = pd.read_csv(url)
DJI['date'] = pd.to_datetime(DJI['Date'])
DJI = DJI.drop(["Date"], axis = 1)

url = "https://raw.githubusercontent.com/nytimes/covid-19-data/master/us.csv"
COVID = pd.read_csv(url)
COVID['date'] = pd.to_datetime(COVID['date'])
COVID
```

Out[62]:

	date	cases	deaths
0	2020-01-21	1	0
1	2020-01-22	1	0
2	2020-01-23	1	0
3	2020-01-24	2	0
4	2020-01-25	3	0
...	...	...	...
1019	2022-11-05	97495251	1068517
1020	2022-11-06	97501890	1068522
1021	2022-11-07	97540189	1068659
1022	2022-11-08	97561206	1068872
1023	2022-11-09	97654808	1069792

1024 rows × 3 columns

Part A

The series `COVID['cases']` is essentially a `numpy` array containing the cumulative case counts over time. The COVID19 case data is cumulative, but we would like to see the number of new cases per day (i.e. as in this kind of plot). Check the documentation for the `np.diff` function and figure out what it does. Use it appropriately to construct a new array, called `per_day`, giving the number of new cases per day. Then, make a new array called `per_day_date` that gives the appropriate date for each case count. In particular, you will need to ensure that `per_day` and `per_day_date` have the same shape.

In [63]:
```
# your solution here
import numpy as np

per_day = np.diff(COVID['cases']) #making array per_day where each entry is the difference in COVID cases
                                  #from the last day
per_day_date = COVID['date'][1:]  #array with the appropriate date for each diff per day count
                                  #exclude the first day since you do not have a change from before it
                                  #up until and including last date
```

```
print(per_day.shape) #checking that both arrays have the same shape
print(per_day_date.shape)
```

```
(1023,)
(1023,)
```

Part B

Create a figure with two very wide axes, one on top of the other (i.e. two rows, one column). Use the `sharex` argument of `plt.subplots()` to ensure that these two plots will share the same horizontal axis.

Then:

1. On the upper axis, plot the Dow Jones Industrial Average over time. For the horizontal axis use `DJI['date']`; the for the vertical use `DJI['Close']`.
2. On the lower axis, plot the variables `per_day_date` and `per_day` to visualize the progress of the COVID19 pandemic over time. Use a different color for the trendline.

Give your plot horizontal and vertical axis labels.

In [64]:
```
# your solution here
# modify this block in the remaining parts of the problem

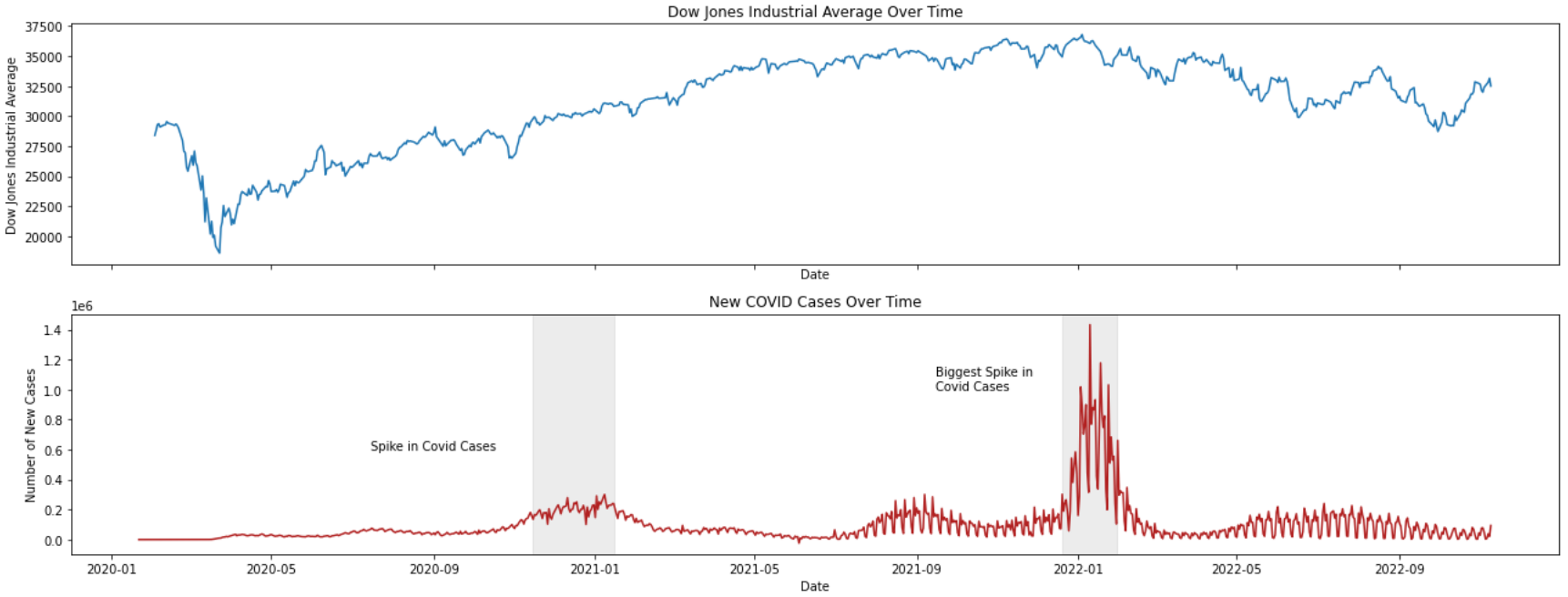
fig,ax=plt.subplots(2,1,figsize=(18,7),sharex=True) #creating two subplots with large width that share x-axis
                                                    #two rows, one column, figsize 18 x 7

ax[0].plot(DJI['date'],DJI['Close']) #on top plot, plot DJI date on x-axis and DJI average on y-axis
ax[1].plot(per_day_date,per_day, color="firebrick") #bottom plot: plot date on x-axis/diff in cases on y-axis
ax[1].axvspan(datetime.datetime(2020,11,15), #creating a gray shade on bottom plot from 11/15/20 to 1/15/2021
              datetime.datetime(2021,1,15),  #with transparency .15
              alpha = .15,
              color = "gray")
ax[1].text(datetime.datetime(2020,7,15), #labeling the above shade as a spike in cases
           600000,                       #position at 7/15/20 with vertical position 600,000
           "Spike in Covid Cases")
ax[1].axvspan(datetime.datetime(2021,12,20), #creating another gray shade on bottom plot from
              datetime.datetime(2022,1,30),  #12/20/2021 to 1/30/2022 with transparency .15
              alpha = .15,
              color = "gray")
ax[1].text(datetime.datetime(2021,9,15), #labeling the above shade and positioning at
           1000000,                      #9/15/2021 and vertical position 1,000,000
```

```
            "Biggest Spike in\nCovid Cases")

#labeling x and y axis and overall plots: first top, then bottom
ax[0].set(xlabel="Date",ylabel="Dow Jones Industrial Average",title="Dow Jones Industrial Average Over Time")
ax[1].set(xlabel="Date",ylabel="Number of New Cases",title="New COVID Cases Over Time")

plt.tight_layout() #making plots look nicer
```



Part C

The command

```
ax[0].axvspan(datetime.datetime(2020,6,1),
              datetime.datetime(2020,6,30),
              alpha = .3,
              color = "gray")
```

will add a simple rectangular shade which can be used to highlight specific portions of a time-series. In the given code, this shade runs through the month of June 2020. Add at least two such rectangular shades to your figure corresponding to important time intervals. You can put two shades on one axis, or one on each. If you're not sure what time periods are important, just

choose intervals at random. Feel free to modify the color and transparency as desired. You can modify your figure code from Part B -- no need for copy/paste.

Part D

The command

```
ax[0].text(datetime.datetime(2020,9,15),
           22000,
           "penguins?\npenguins!")
```

will add a fun text annotation to your plot, with the first letter in horizontal position corresponding to September 15th, and at vertical position 22,000. Annotate each of your shaded regions with a few words describing their significance. Again, just modify your Part B code.

Part E

Add an overall title, spruce up your axis labels, and add anything else you think will make the plot look good. Again, you can just modify your Part B code, without copy/paste.

Then, submit a job application at www.FiveThirtyEight.com and show Nate Silver your cool data visualization.

In [ ]: